Clasificación de la calidad de frutas mediantes redes neuronales convolucionales

Sebastian Perez Garcia
A01707346
Instituto Tecnológico y de Estudios Superiores de Monterrey

ABSTRACT

Este trabajo desarrolla un modelo de aprendizaje profundo para clasificar seis tipos de fruta y su calidad a partir de fotografías del dataset FruitNet. Tras limpiar los datos y dividirlos por sesiones de captura, se partió una CNN base con sobreajuste y se aplicaron mejoras incrementales. El mejor modelo está basado en transfer learning y fine-tuning de MobileNetV2, alcanzando un 83.3% de exactitud en el conjunto de pruebas y puede generar predicciones por consola o mediante una interfaz gráfica.

1. Introducción

1.1 Contexto General

Uno de los problemas más relevantes en el sector de los sistemas agroalimentarios hoy en día es la pérdida y el desperdicio de alimentos. Según la Organización de las Naciones Unidas para la Alimentación y la Agricultura (FAO), cerca del 14% de los alimentos producidos en el mundo se pierden entre la cosecha y la venta, es debido a eso que las frutas y las hortalizas son de las categorías más afectadas por su naturaleza. [1]. Lamentablemente, el impacto para los productores y distribuidores es económicamente desgastante debido a que estas pérdidas contribuyen al desperdicio de agua, energía y tierra cultivable.

Por otra parte, uno de los puntos críticos dentro de la cadena de suministro es la inspección de calidad. De hecho según estudios recientes sobre logística e infraestructura comercial [2] menciona que hoy en día muchos de los centros de mercados, empaques, y de abastecimiento. Elaboran la separación entre las frutas de buen estado y de malo de manera manual, a consecuencia de esto este proceso es lento, depende del criterio y la fatiga de cada inspector, y es difícil escalar a grandes volúmenes.

Gracias a las nuevas tecnologías ya empiezan a existir formas de automatizar este proceso. Por ejemplo, las redes neuronales convencionales mejor conocidas como CNN han demostrado un alto desempeño en tareas por el área de clasificación de imágenes, debido a que aprenden directamente de los pixeles las características relevantes, como es el caso de manchas, cambio de colores, arrugas, o moho. El beneficio de dicha es que no es necesario definirlas manualmente, es por eso que es una gran herramienta en cuanto a la identificación de defectos superficiales en la fruta.

Dentro de este documento se propone un modelo de aprendizaje profundo basado en CNN que a partir de una fotografía, se identifica el tipo de fruta y determina su estado de calidad, cabe mencionar que este modelo se entrena con imágenes reales de 6 frutas (Manzana, plátano, guayaba, lima, naranja y granada) tomadas con teléfonos celulares en condiciones no controladas, por lo que se acerca a un escenario de uso adecuado.

1.2 Descripción del dataset

Dentro del conjunto de datos seleccionado es utilizado *FruitNet:Indian Fruits with Quality* [3], el cual está disponible desde la plataforma de Kaggle. Por lo analizado se puede observar que este dataset contiene fotografías reales de seis frutas comunes en India, las cuales son manzanas, plátanos, guayabas, limas, naranjas y granadas. Como mencionado en la sección anterior las

imágenes fueron tomadas con teléfonos celulares en condiciones no controladas, esto se puede observar debido a que las fotos tienen distintos fondos, iluminación que varía, y diferentes ángulos. Cabe mencionar que, los nombres de las fotografías parecen que mencionan un rango de fecha de cuándo fueron tomadas el cual va desde el 2019 al 2020.

Adentrándonos más a las carpetas del dataset podemos observar que el dataset está organizado en tres capas principales: Para empezar tenemos la carpeta de **Bad Quality_Fruits**, la cual son frutas en mal estado, con manchas, golpes, moho, arrugas o descomposición. Por otro lado está **Good Quality_Fruits**, la cual son frutas que se encuentran en buen estado sin daños visibles. Finalmente, se encuentra **Mixed Quality_Fruits**, las cuales son fotografías que contienen ambos tipos de frutas, tanto buenas como malas. Dentro de cada carpeta podemos observar que se encuentran subcarpetas por cada fruta, cabe mencionar que las imágenes se encuentran en el formato de JPG con resoluciones de 256x256 px y 256x192 px, a continuación en la tabla 1.2.1 se puede demostrar una comparación de la distribución por cada fruta:

Fruta	Buena Calidad	Mala Calidad	Mixta
Manzana	1,149	1,141	113
Plátano	1,113	1,087	285
Guayaba	1,152	1,129	148
Lima	1,094	1,085	278
Naranja	1,216	1,159	125
Granada	5,940	1,187	125
Total	11,664	6,788	1,074

Tabla 1.2.1 - Distribución por fruta

En total el dataset contiene 19,526 imágenes. Cabe mencionar, que a la hora de revisar el dataset se identificaron algunas características que requieren tratamiento antes del tratamiento y que se detallarán en el área del ETL.

1.3 ¿Qué se está prediciendo?

El objetivo de este modelo es que a partir de una fotografía de una fruta se prediga la categoría de la fruta y su calidad (mal estado, buen estado). Debido a esto ambas características se combinan en una sola etiqueta, por lo que cada imagen pertenece a alguna de las siguientes 12 clases mostradas en la tabla 1.3.1:

Fruta	Buena Calidad	Mala Calidad
Manzana	Apple_Good	Apple_Bad
Plátano	Banana_Good	Banana_Bad
Guayaba	Guava_Good	Guava_Bad
Lima	Lime_Good	Lime_Bad
Naranja	Orange_Good	Orange_Bad
Granada	Pomegranate_Good	Pomegranate_Bad

Tabla 1.3.1 -Clasificación de etiquetas

Cabe destacar que la entrada del modelo sería una imagen RGB de la fruta, obviamente redimensionada a un tamaño fijo, para mantener un estándar y como salida se esperaría un vector de 12 probabilidades, una por clase, la cual sería generado por una capa de *softmax*. Gracias a esto el modelo identificaría la clase con la mayor probabilidad, la cual terminaría siendo la predicción del modelo y también sacando un valor de la seguridad, en otras palabras lo seguro que está el modelo está en la clasificación. Por ejemplo, Guava_Bad: 90%.

Un dato importante es que en este caso se eligió combinar la fruta y la calidad en una sola etiqueta en vez de solo predecir si es buena o mala, esto es debido a que el aspecto de una fruta en mal estado depende del tipo de fruta. Por ejemplo, un plátano maduro con manchas oscuras, se ve distinto a una granada con moho o incluso a una lima arrugada. Gracias a estos aspectos pequeños le ayuda al modelo a aprender los defectos propios de cada fruta, por lo que le permite analizar con más detalle los errores del modelo. Un ejemplo de la utilidad de esto, es poder observar si el modelo confunde una lima con una naranja o si confunde una manzana buena con una mala, los cuales son errores de naturaleza muy distintos entre sí.

1.4 Tipo de problema y arquitectura elegida

En este caso y como observado en la sección anterior podemos concluir que es un problema de clasificación multiclase supervisada. Se llegó a esta conclusión debido a que la salida es una categoría discreta y no un valor numérico continuo. Por otro lado, es considerado como multiclase debido a que tiene 12 categorías posibles y cada imagen pertenece a una. Por el contrario, es considerado como supervisado debido a que cada imagen del conjunto de entrenamiento cuenta con su etiqueta correcta, debido a la carpeta en la que se encuentran.

Es por eso que se decidió elegir una red neuronal convolucional mejor conocida como CNN, al principio se pensó utilizar un MLP, mejor conocido como un perceptrón de multicapas, sin embargo al analizar se observaron algunas razones por las cuales no es óptimo. Por ejemplo, que para pasar un MLP se tiene que aplanar la imagen en un vector y aquí es en donde se pierde la relación entre pixeles vecinos, la importancia de esto es que la relación de un pixel y el pixel de alado son tan independientes como si fueran dos pixeles de esquinas opuestas. Otra razón es que se necesitan demasiados parámetros, esto es debido a que una imagen de 224 x 224 pixeles con 3 canales de color tiene alrededor de 150,528 valores de entrada. De hecho, una sola capa densa de 512 neuronas conectada a esta entrada necesitaría más de 77 millones de pesos, lo cual hace el entrenamiento lento y esto hace que sea muy propensa al sobreajuste.

Por el contrario, las CNN resuelven estos problemas debido a que cada filtro convolucional analiza una pequeña región de la imagen a la vez, esto permite detectar patrones locales como son los bordes, cambios de color o textura. Otra razón son los pesos compartidos esto es debido a que el mismo filtro recorre toda la imagen. Es por eso que una mancha de moho se detecta de igual manera que el centro o una orilla de la fruta, y el número de parámetros se reduciría de manera drástica. Para finalizar tiene la jerarquía de características la cual significa que las primeras capas aprenden los patrones simples como son los bordes, y colores, posteriormente las capas más profundas utilizan lo aprendido de las primeras capas y lo combinan con patrones más complejos como son las texturas, zonas de deterioración, formas de la fruta, entre otras características.

Gracias a las razones anteriores hacen que las CNN sean el estándar para la clasificación de imágenes y encajan de manera directa con el problema, el cual es detectar los defectos visuales en la superficie de la fruta.

Cabe mencionar que el modelo es incremental, a lo que me refiero es que primero se entrena la CNN, posteriormente se aplican las mejoras sucesivas y finalmente se evalúa el uso de TransferLearning con una red pre entrenada, la cual aprovecha las características visuales aprendidas previamente en millones de imágenes.

2. Extracción, Transformación y Carga de Datos (ETL)

2.1 ¿Qué es ETL?

ETL es un acrónimo de “Extract, Transformación y Carga de datos” es un proceso mediante el cual los datos se obtienen de una o varias fuentes, se limpian y transforman a un formato más analizable. Finalmente, se cargan en un destino en donde se pueden analizar o modelar. El ETL es un marco de trabajo estándar en cuanto a la ingeniería de datos esto es debido a que ayuda a preparar la información desde antes de cualquier análisis o entrenamiento de modelos. [4]. El ETL tradicionalmente se compone en tres etapas:

Extracción: Es en donde se obtienen los datos crudos desde la fuente original, en esta área no se hace ningún tipo de cambio a la data.

Transformación: Es en donde se limpian, validan y reestructuran los datos. También, es en donde se manejan los valores nulos, la eliminación de duplicados y la codificación de variables categóricas. Todo esto es para que los datos estén listos para proseguir con el área de análisis.

Carga: Es en donde se almacena el resultado final en un formato listo para ser consumido por las siguientes etapas del proyecto.

Aunque el ETL surgió para los datos empresariales, el mismo marco de tres etapas aplica directamente a cualquier proyecto de ciencias de datos, esto se debe a que ayuda a tener un proceso en cuestión de los datos ya que, en muchos casos los datos no son consistentes, incluyen errores y tienen formatos diferentes causando inconsistencias dentro de los modelos. Es por eso la importancia de aplicar un ETL en cualquier modelo que se esté trabajando con datos, ya que ayuda a evitar muchos de estos problemas que pueden afectar la manera en la que funciona nuestro modelo. A continuación la sección 2.2 -2.5 documenta cómo se aplicó cada etapa del dataset en el proyecto.

2.2 Extracción

Como mencionado anteriormente, los datos se obtuvieron de la plataforma de Kaggle, dentro de ella contamos con aproximadamente 19,500 imágenes en formato de JPG, los cuales están organizados en carpetas según la calidad y el tipo de fruta.

Para la etapa de extracción se realizaron los siguientes pasos: Para empezar el archivo se tuvo que comprimir para poder ser almacenado en Google Drive, esto para facilitar el trabajo y no tenerlo que estar descargando por cada sesión de Colab, por otro lado se tuvo un montaje de Google Drive dentro de Colab para que al inicio de cada sesión Colab pudiera detectar el zip que se encuentra dentro de la Unidad y finalmente, se tuvo la descompresión en el almacenamiento local de Colab, en donde el archivo se descomprime en /content/fruitnet y no se lee directamente desde Drive,debido a que leerlo desde Drive es demasiado lento, es por eso que se realizo un metodo para que se leyera en el disco local de Colab para que fuera más rápido.

Posteriormente una vez que esté descomprimido el script 01_ETL_CFRN recorre las carpetas de Good Quality_Foods y Bad Quality_Foods y así construyendo una tabla con la información de cada imagen, como podemos observar en la tabla 2.2.1:

Campo	Descripción	Ejemplo
ruta	Ubicación relativa del archivo	Bad Quality_Fruits/Apple_Bad/IMG_7648_3647.JPG
clase	Etiqueta combinada de fruta y calidad	Apple_Bad
fruta	El tipo de fruta	Apple

calidad	El tipo de calidad de la fruta	Bad
archivo	Nombre del archivo	IMG_764832_3647.JPG

Tabla 2.2.1 - información por imagen

Cabe mencionar, que durante esta etapa se lee la fecha y hora de captura de cada fotografía. Para poder obtener esta información se realizó utilizando los metadatos EXIF de la imagen en el caso de que estuvieran disponibles. Por el contrario, si estos metadatos no estaban disponibles se obtiene desde el nombre del archivo, debido a que hay muchos casos en donde el nombre del archivo incluye la fecha, esto es notorio como en la imagen '20190809_115439.jpg', el cual fue observado que su nombre mencionada la fecha y hora de captura de dicha imagen. Conviene resaltar que la fecha de captura no se utilizó como variable de entrada sino que se utilizó para identificar las fotografías que fueron tomadas en racha y así evitando la fuga de información al dividir los datos, esto será explicado más profundamente en la sección 2.5, la cual habla de la división de los datos.

En total se extrajeron 18,452 imágenes de las carpetas de buena y mala calidad, cabe mencionar que las 1,074 imágenes de la carpeta de calidad mixta no fueron incluidas en el conjunto de entrenamiento, debido a que contiene frutas de ambas calidades y no se tiene una sola etiqueta, de hecho estas serán reservadas para las pruebas cualitativas del modelo final

2.3 Revisión de Calidad

Antes de transformar los datos se hizo una revisión de la calidad, en donde se combinó la inspección visual de muestra de cada clase, el conteo de archivos por carpeta, el análisis de los nombres de los archivos y los metadatos de las imágenes. El principal objetivo de esto fue identificar los problemas que pudieran influir en las métricas del modelo o afectar el entrenamiento.

Para empezar se encontraron copias aumentadas en la clase de Pomegrade_Good (Granada), esta clase cuenta con 5,940 imágenes, unas cincos veces más que cualquier otra clase ya que todas se encuentran en un rango entre 1,085 y 1216. Al revisar los nombres se encontró que 4,752 de ellas cuentan con un sufijo numérico de cinco dígitos, como por ejemplo "20190820_143733_22996.jpg", esto provocó sospecha y al compararlas visualmente con la fotógrafa original (20190820_143733.jpg) mostró que eran la misma imagen solo rotada o inversa, en otras palabras eran copias generadas para el aumento de los datos y no fotografías nuevas, de hecho esta clase solo cuenta con 1,188 fotografías distintas. El problema de esto es que puede crear un desbalance de las clases debido a que el modelo vería 5 veces más ejemplos de la granada buena que de cualquier otra clase y esto favorecía en sus predicciones a dicha clase. Por otro lado, existiría una fuga de información debido a que en el caso de que una fotografía queda en entrenamiento y su copia rotada en prueba, el modelo sería evaluado con una fotografía que ya conoce y esto a consecuencia infla la precisión.

Otro caso encontrado fue que al analizar las fechas se observó que las fotografías fueron tomadas de forma casi continua aproximadamente una por segundo, girando la misma fruta o moviendo ligeramente la cámara, lo malo de muchas imágenes consecutivas es que son básicamente idénticas. De hecho si estas imágenes se reparten al azar entre las secciones de prueba, entrenamiento y validez causaría la misma fuga de información que se identificó en las copias aumentadas, debido a que el modelo se evalúa con fotos casi iguales a las que vio al entrenar.

Otro factor es que podemos concluir que las imágenes se tomaron con al menos 5

diferentes teléfonos debido a que como muestra la tabla 2.3.1 enseña los distintos formatos encontrados:

Formato	Ejemplo	Origen de la fecha
IMG_AAAAMMDD_HHMMSS.jpg	IMG_20190824_175205.jpg	Metadatos EXIF
IMGAAAAMMDDHHMMSS.jpg	IMG20200728175856.jpg	Metadatos EXIF
AAAAMMDD_HHMMSS.jpg	20190809_115439.jpg	Nombre del archivo (Sin EXIF)
IMG_NNNN.JPG	IMG_0111.JPG	EXIF, o número de secuencia de la cámara

Tabla 2.3.1 - Formatos encontrados

De hecho la mayor parte de las imágenes conservan la fecha de su captura en sus metadatos, solo 1,577 imágenes no se les encontró en los metadatos, sin embargo se obtuvo mediante el nombre del archivo. Para finalizar se encontró que en 793 imágenes de Guava_Good no había fecha disponible por ningún medio, para estas se utilizó el número consecutivo que les asigna la cámara para referenciar el orden en las que se tomaron.

Por otro lado, se calculó el hash MD5 de cada archivo para poder detectar las imágenes idénticas, sin embargo, no se encontraron duplicados exactos ni dentro, como fuera de las clases. Cabe mencionar que las copias encontradas de la granada no aparecen en esta sección debido a que gracias a la rotación sus bytes son distintos a los de la original. Al continuar inspeccionando se encontró que todas las etiquetas son consistentes con el contenido de las imágenes mostrando como las frutas de mala calidad muestran defectos y las de buena calidad están en perfectas condiciones. Cabe mencionar que las 1,074 imágenes que se encuentran en Mixed Quality_Fruits contienen frutas de buena y mala calidad en la misma fotografía y es por eso que no se pueden asignar a una sola clase. Otro aspecto encontrado en esta carpeta es que existen inconsistencias de nombres ya que utiliza “Lemon” en lugar de “Lime” cómo referenciado en las otras carpetas y el nombre de la carpeta esta incompleto siendo Qualit en lugar de Quality. Es por eso que estas imágenes fueron excluidas en el entrenamiento y serán utilizadas para las pruebas cualitativas.

Para finalizar, se detectó que las imágenes vienen reducidas a una resolución baja de 256 x 256 o de 192 x 256 píxeles según sea la orientación de la foto, como en este caso estamos trabajando con una red neuronal se necesita que estas entradas sean de un tamaño fijo y es por eso que será necesario redimensionarlas a un tamaño fijo.

2.4 Limpieza y Transformación

Tomando en cuenta lo observado en la sección 2.3 se aplicaron las siguientes transformaciones.

Para empezar se eliminaron las copias aumentadas, para cada fotografía se identificó el nombre base y se eliminaron las versiones que tenían los prefijos y en caso de que solo existían copias se conservaba la primera de ellas.Gracias a esto se eliminaron 4,752 imágenes y la clase de Pomegrade_Good paso de 5,940 a 1,188 imagenes entrando en el rango del resto de las imágenes, las otras 11 clases no fueron modificadas en esto.

Por otro lado se realizó la agrupación en sesiones de captura. Para evitar que fotografias casi idénticas quedarán repartidas entre los dos conjuntos de entrenamiento y prueba las imágenes de cada clase fueron ordenados por su fecha de captura y posteriormente fueron agrupadas por sesiones, en donde dos fotos consecutivas pertenecen a la misma sesión si se toman con 10 segundos o

menos de diferencia, en el caso de que sea mauor comienza una sesión nueva. Cabe mencionar que el umbral fue seleccionado debido a que al ritmo que se tomaban las fotos siendo de aproximadamente una por segundo un umbral pequeño tendría una misma fruta fotografiada en distintos ángulos partida en varias sesiones, por el contrario uno grande tendría sesiones de frutas distintas en grupos demasiados grandes. Todo esto dificulta repartir los datos en las proporciones deseadas, dicho esto con el umbral de 10 segundos se obtuvieron 588 sesiones con una mediana de 10 imágenes por sesión.

Para la división de los datos de entrenamiento, validación y pruebas se dividieron de la siguiente manera:

Entrenamiento cuenta con el 70% para ajustar los pesos del modelo. Por otro lado, la validación cuenta con 15% para comparar versiones del modelo, ajustar hiperparametros y detectar sobreajuste durante el entrenamiento y finalmente, pruebas con un 15% para medir el desempeño final. Esta distribución se hizo debido a la sesión y la forma estratificada por clase, cabe mencionar que todas las imágenes de una sesión van al mismo conjunto y cada clase se reparte por separado en las proporciones de 70/15/15. Para lograr lo dicho las sesiones de cada clase se mezclan de manera aleatoria con un semilla fija de 42, se ordenan de mayor a menor tamaño y cada una se asigna al conjunto que más lejos está de su proporción objetivo.Como podemos observar en la tabla 2.4.1 se muestran los resultados obtenidos:

Clase	Imágenes	Sesiones	Train	Val	Test
Apple_Bad	1,141	35	800	171	170
Apple_Good	1,149	165	804	173	172
Banana_Bad	1,087	20	761	163	163
Banana_Good	1,113	24	779	166	168
Guava_Bad	1,129	30	790	169	170
Guava_Good	1,152	54	806	173	173
Lime_Bad	1,085	32	759	163	163
Lime_Good	1,094	22	768	163	163
Orange_Bad	1,159	26	811	175	173
Orange_Good	1,216	60	851	183	182
Pomegranate_Bad	1,187	38	831	178	178
Pomegranate_Bad	1,188	82	832	178	178
Total	**13,700**	**588**	**9,592**	**2,055**	**2,053**

Tabla 2.4.1 - Resumen por clase

Como podemos observar las 12 clases quedaron balanceadas y la proporción final coincide con el 70/.15/15 buscado.

Para finalizar, como las imágenes tienen tamaños diferentes todas se redimensionada a 224 x 224 píxeles al cargarlas, este tamaño fue elegido debido a que es la entrada estándar de las redes pre entrenadas que se utilizarán más adelante y esto permite comparar todos los modelos con la misma entras. Cabe mencionar que la transformación no modifica los archivos en disco sino que se aplican al cargar las imágenes como se describe en la siguiente sección.

2.5 Carga

Dentro de esta sección, una vez transformados los datos, se copia cada imagen a una estructura de carpetas organizadas por conjuntos y por clases (/content/data/test, /content/data/train y /content/data/val), cada uno de ellas cuenta con 12 subcarpetas una por clase. Por otro lado se generó un archivo llamado “Splits.csv”, el cual registra para cada una de las 13,700 imágenes su clase, su sesión de captura y al conjunto al que fue asignado para poder verificar que no haya fuga de información y que la división quede documentada.

Gracias a esta estructura las imágenes se cargan con la función de image_dataset_from_directory de keras, la cual asigna la etiqueta de cada imagen según la carpeta en la que se encuentra y al cargarlas las imágenes se redimensionan a 224 x 224 píxeles, cabe mencionar que las etiquetas se representan con codificación one-hot, un vector de 12 posiciones con un 1 en la clase correcta la cual es el formato requerido para la función de costo de entropía cruzada categórica. Gracias a todo esto la carga conformó 9,592 imágenes de entrenamiento, 2,055 de validación y 2,053 de prueba, distribuidas en 12 clases. No solo eso sino que las imágenes se entregan al modelo en lotes de 32 para evitar cargar todo el conjunto de memoria a la vez y permitir actualizar los pesos después de cada lote.

Por otro lado, el conjunto de entrenamiento se mezcla de manera aleatoria en cada época para que el modelo no aprenda del orden de las imágenes, mientras que por el contrario validación y prueba conservan un orden fijo para que las predicciones puedan compararse de manera directa con su etiqueta. Como se puede notar también se utilizan métodos como el “prefetch” para preparar el siguiente lote mientras la GPU procesa el actual así reduciendo el tiempo de espera durante el entrenamiento. Cabe mencionar que la normalización de los píxeles al rango [0,1] se realizará dentro de la sección del modelo.

3. Análisis Exploratorio de Datos (EDA)

3.1 ¿Qué es el EDA?

El análisis Exploratorio de datos también conocido como el EDA por sus siglas en inglés. Es un enfoque para analizar conjuntos de datos que consiste en resumir sus características principales, frecuentemente mediante métodos visuales antes de ser aplicados en cualquier modelo estadístico. El término fue definido por Jhon Tukey en 1977 en su libro, en donde describe al EDA como una forma de dejar que los datos hablen.Por otra parte, el EDA busca en describir patrones, detectar anomalías, definir suposiciones, y generar nuevas preguntas a partir de los datos obtenidos.[5]

En la práctica el EDA involucra examinar la distribución de cada variable, identificar los valores atípicos, los cuales podrían causar errores de captura o entender la composición de las variables categóricas, y medir la relación entre variables mediante correlaciones. De hecho durante esta sección se hablará de las distribuciones, los valores atípicos, variables categóricas y las correlaciones.

El EDA cumple con el propósito de ir más allá al describir los datos, ya que es el paso en donde informa decisiones posteriores del proyecto. La importancia del EDA es que nos ayuda a validar, descartar los datos y variables presentes dentro de nuestro proyecto, las cuales nos ayudan a construir nuestro modelo.

3.2 Distribución de clases

El primer aspecto analizado fue la distribución de imágenes entre las 12 clases,

como se puede observar en la figura 3.2.1 se comparó el dataset original con el resultado de la limpieza. Se puede observar que en el dataset original 11 de las 12 clases tenían entre 1,085 y 1,216 imágenes, excepto por Pomegranade_Good, el cual contaba con 5,940 imágenes y daba una relación de 5.47 entre la clase más grande y las más pequeña. Como fue mencionado anteriormente en la sección 2.3, la diferencia no se debía a que hubieran más fotografías de granadas en buen estado, sino que 4,752 de ellas solo eran copias rotadas y volteadas de las mismas imágenes. De hecho si no se hubieran eliminado provocaría fallos en el modelo debido a que dicho hubiera visto cinco veces más ejemplos de esa clase que de cualquier otra cosa y eso le favorece en sus predicciones aun cuando la imagen perteneciera a otra cosa. A continuación se puede observa la figura 3.2.1, la cual muestra lo mencionado:

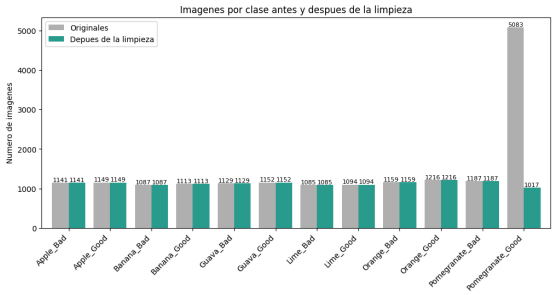


Figura 3.2.1 - Imágenes por clase antes y después de la limpieza.

Después de la limpieza podemos observar como Pomegranate_Good quedó con solo 1,188 imágenes y gracias a esto la relación entre la clase más alta y la más pequeña bajó a 1.12. Gracias a esto el dataset quedó balanceado, en este y en otro aspecto. El otro aspecto mencionado es que antes existían más imágenes de buena calidad que de mala debido a que Pomegranade_Good inflaba las imágenes que existían de buena calidad, gracias a esto tenemos 6,912 imágenes de buena calidad y 6,788 de mala calidad, no solo eso pero también se ajustó que cada clase tuviera un rango de entre 2,179 y 2,375. Haber ajustado este balance ayuda a que la exactitud se vuelva una métrica representativa del desempeño del modelo y hace innecesario aplicar técnicas de compensación como pesos por clase o sobremuestreo. Por el contrario, en un dataset desbalanceado un modelo puede obtener una exactitud alta simplemente por predecir la clase mayoritaria, sin embargo en este caso al haber emparejado toda la cantidad de imágenes entre las clases solo daría alrededor de un ~9% de aciertos.

Por otro lado también se realizó una verificación del reparto entre entrenamiento, validación, y pruebas, en donde se respetan las proporciones de cada clase, como podemos observar en la figura 3.2.2 se puede analizar la división que se realizó por sesiones completas y no por imágenes individuales, cabe mencionar que existía un riesgo de que algunas clases se quedarán con una proporción distinta a la buscada, sobre todo aquellas que contaban con pocas sesiones como es el caso de Banana_Bad con 20 sesiones y Lime_Good con 22. Sin embargo, como se muestra en la figura 3.2.2 todas las clases quedaron con su 70% de entrenamiento, 15% de pruebas y 15% de validación, con una desviación máxima de 0.2 puntos porcentuales.

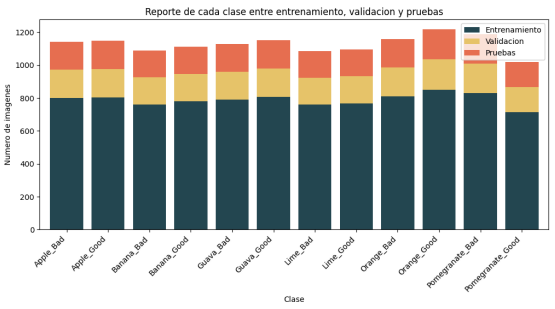


Figura 3.2.2 - Partición de cada clase entre entrenamiento, validación y pruebas

3.3 Tamaños y formatos de imagen

Debido a que una red convolucional requiere que todas sus entradas tengan el mismo tamaño se realizo una revision de resolucion la resolución y el modo de color de las 13,700 imagenes. Gracias a esto se observo que todas las imagenes estan en el modo de

RGB, es decir que tienen tres canales de color. Es por eso, que no hubo necesidad de convertir las imágenes en escala de grises o con canal de transparencia. Como podemos observar en la tabla 3.3.1 se puede observar la resolución, de los 5 diferentes tamaños:

Resolución	Orientación	Imágenes	Porcentaje
256 x 256	Cuadrada	8,320	60.7%
192 x 256	Vertical	3,968	29.0%
256 x 192	Horizontal	1,391	10.2%
144 x 256	Vertical	12	0.1%
256 x 144	Horizontal	9	0.1%

Tabla 3.3.1 - Resolución de imágenes

En todos los casos el lado más largo de las imágenes mide 256 píxeles. Esto indica que los autores del dataset redujeron las fotografías originales del teléfono a un tamaño máximo de 256 pixeles conservando su proporción y que una parte de las imágenes ya estaba recortada en forma cuadrada. Debido a que el tamaño de entrada escogido para el modelo es de 224 x 224, todas las imágenes se redujeron ligeramente y a consecuencia de esto no se necesita ampliar ninguna, lo cual evita perder la nitidez. Lamentablemente, al bajar la resolución también baja el detalle, por lo que a consecuencia defectos muy pequeños quedan representados por unos cuantos píxeles.

Posteriormente, al continuar el análisis se encontró que la orientación de las imágenes no era igual en todas las clases. De hecho, el 72.2% de las imágenes con buena calidad eran cuadradas, por el contrario las de mala calidad contaban con solo un 51.0% de imágenes cuadradas. Por otro lado, lo mismo sucedía en las clases como por ejemplo, Guava_Good y Pomegranade_Good solo contienen 1,000 imágenes cuadradas cada una, mientras que Lime_Good no contaba con ninguna y de hecho la mayoría de sus imágenes eran de modo horizontal. De hecho al redimensionar las imágenes a 224 x 224, las imágenes cuadradas quedan igual, pero las imágenes horizontales y verticales se estiran o se comprimen respectivamente, gracias a que estas deformaciones ocurren con distinta frecuencia en cada clase, brindaría un riesgo al modelo debido que podría aprender a utilizar estas peculiaridades a su favor utilizando la forma deformada de la fruta como pista para clasificarla en vez de fijarse en sus defectos. Este riesgo es retomado en la sección 3.6 junto con los demás posibles sesgos, para finalizar una alternativa para eliminar este problema es redimensionar las imágenes agregando bordes para conservar su proporción original.

3.4 Muestras visuales por clase

Por otra parte, para completar el análisis numérico, se seleccionaron al azar 5 imágenes del conjunto de entrenamiento de cada clase, esto se puede observar en la figura 3.4.1. Dentro de esta revisión se permitió confirmar que las etiquetas correspondieran al contenido de las imágenes. Por ejemplo, las frutas de mala calidad muestran defectos visibles, ya sea porque tienen manchas negras, zonas hundidas, zonas putrefactas, moho, piel arrugada, entre otro tipo de deformaciones. Por otro lado, las frutas de buena calidad muestran condiciones perfectas.

Las muestras también reflejan la variedad de condiciones en las que se tomaron las fotografías. Para empezar, el número de frutas por imagen varía de una sola hasta grupos de varias, en muchas de las fotos también es notorio que aparece una mano sosteniendo la fruta y de hecho también la iluminación cambia de una imagen a otra, haciendo que las condiciones cambien de foto en foto. Gracias a esta gran variedad, ayuda a que el modelo reconozca las frutas y sus defectos en diferentes situaciones acercándo lo

a un escenario más real. Otro caso que cabe mencionar es lo que fue establecido en la sección 2.3 en donde se notan fotos muy similares, solo cambiando el ángulo, lo cual confirma la presencia de las fotografías en rafagas y justifica nuestra división por secciones.

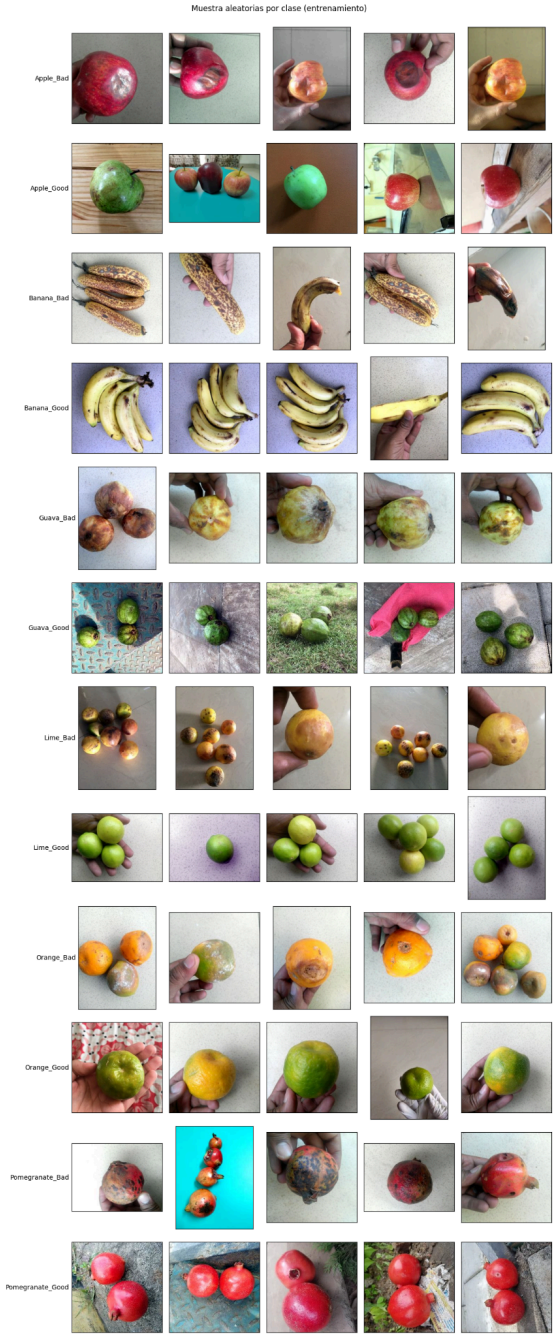


Figura 3.4.1 - Muestra aleatoria por clase del conjunto de entrenamiento

Lamentablemente, esta revisión también reveló aspectos que podrían afectar la dificultad de la clasificación. Para empezar, la frontera entre buena y mala calidad no siempre es nítida debido a que por ejemplo algunos plátanos de buena calidad como mostrados en la figura 3.4.1 tienen pequeñas manchas cafés, las cuales son referencia de que los plátanos están madurando, por lo que el modelo tendría que distinguir entre las manchas de maduración y las de putrefacción, lo cual sería un poco más complejo. Otro posible riesgo es que el color no baste para identificar la fruta, como se puede observar existen naranjas de buena calidad que están del color verde y esto hace que se parezcan a las limas, por lo que podría pasar que el modelo se confunda entre ambas frutas. Para finalizar, los fondos no están distribuidos de la misma manera entre diferentes calidades, por ejemplo podemos notar un patrón en donde las frutas de mala calidad se encuentran usualmente en pisos claros y uniformes, mientras que las de buena calidad se encuentran sobre todo en fondos más variados, ya sea madera, pasto, tela, etc. Este último caso también podría hacer que el modelo se aprenda el fondo y correlacione el fondo con la calidad de la fruta en vez de enfocarse en su calidad, por lo que esto será analizado en la sección 3.6.

3.5 Sesiones de captura

Como se explicó en la sección 2.4, las imágenes se agruparon en sesiones de captura básicamente conjuntos de fotografías de una misma clase tomadas con 10 segundos o menos de diferencia entre sí. En total se tienen 588 sesiones. Como se puede observar en la tabla 3.5.1 e Figura 3.5.2 se muestran las distribuciones de tamaños.

Estadístico	Imágenes por session
Mínimo	1.0
Primer Cuartil	4.0
Mediana	10.0

Promedio	23.3
Tercer Cuartil	27.0
Máximo	472.0

Figura 3.5.1 - Muestra de estadísticas descriptivas de las sesiones de captura

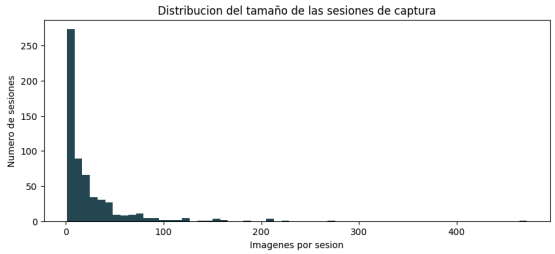


Figura 3.5.2 - Distribución de tamaño de las sesiones de captura

La distribución está fuertemente sesgada a la derecha, la mitad de las sesiones tiene 10 imágenes o menos, pero existen algunas sesiones muy grandes de cientos de fotografías tomadas de forma continua que elevan el promedio a 23.3 imágenes y finalmente se tiene una desviación estándar del 38.5. Gracias a esto, podemos notar lo que se ha mencionado de que muchas de las imágenes de este dataset fueron tomadas de manera secuencial, girandolas o moviendo ligeramente la cámara entre tomas.

Aunque el dataset contenga 13,700 imágenes el número de escenas realmente distintas es mucho menor debido a que las imágenes de una misma sesión comparten la fruta, el fondo, y la iluminación. Es debido a esto que es indispensable dividir los datos por sesión. En el caso de que las imágenes se hubieran repartido al azar, casi todas las sesiones hubieran quedado divididas entre entrenamiento y prueba, y gracias a esto el modelo hubiera sido evaluado con fotografías casi idénticas a las que vio durante su entrenamiento y esto sesgaría el resultado.

Otro caso que cabe mencionar es como el número de sesiones varía entre clases, por ejemplo desde 20 en Banana_Bad, y 22 en Lime_Good, hasta un 165 en Apple_Good. En las clases de pocas sesiones los conjuntos de validación y prueba se forman a partir de unas cuantas escenas, por lo que el modelo se evalúa con menos variedad de frutas, fondos e incluso iluminación comparados con los de varias sesiones. Esto causaría que haya un peor desempeño en estas clases debido a que depende de pocas escenas.

3.6 Posibles sesgos

Para empezar los modelos de aprendizajes profundos aprenden de cualquier característica que le ayude a distinguir las clases del conjunto de entrenamiento, aunque no tenga relación con el problema real. Es por eso que si el dataset las frutas de mal estado están fotografiadas con el mismo teléfono, o tienen el mismo fondo el modelo puede aprenderse a reconocer el teléfono o incluso el fondo en lugar de los defectos de la fruta. Es debido a esto que, el modelo mostraría buenos resultados en cuanto a las imágenes sin embargo, a la hora de aplicarlo fuera de estas condiciones se notaría un cambio drástico. Debido a esto, se analizaron cuatro posibles fuentes de sesgo: el teléfono con el que se tomó la fotografía, el periodo de captura, finalmente el fondo de la imagen.

Para empezar, el primer sesgo mencionado será el teléfono de captura. Gracias a los metadatos EXIF se identificaron 5 teléfonos distintos, además de un grupo de imágenes sin metadatos. En la figura 3.6.1 se puede observar la distribución de los teléfonos con las clases, como se puede observar se encuentran 2,370 imágenes sin metadatos EXIF y se identificó que el teléfono más utilizado para la captura de las fotos es el Z2 Plus con 6,395 imágenes tomadas como se puede observar en la figura esta cuento con fotos de todas las clases excepto de Pomegranate_Good, Guava_Good, Bannana_Good, y Apple_Good. Por otro lado, se puede observar que Realme 5 Pro, cuenta con fotografías de todas las clases y SM-G950F solo cuenta con 321 fotografías de

Apple_Good. Cabe mencionar que el modelo puede identificar qué imagen es sacada del teléfono debido a que cada cámara procesa el color, contraste y la nitidez de diferente manera y el modelo puede aprender a reconocer el estilo de cada uno de estos dispositivos y asociarlo con el tipo de calidad.

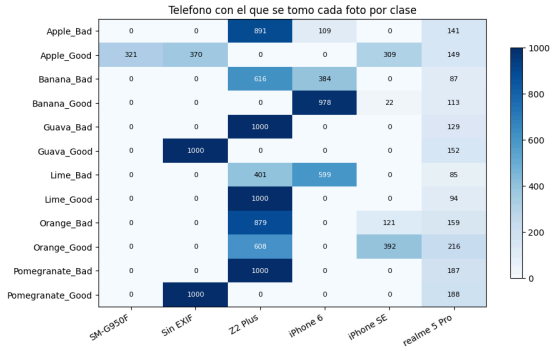


Figura 3.6.1 - Distribución de telefono de captura

Por otra parte, el periodo de captura como se puede mostrar en la figura 3.6.2 el dataset se construyó en dos etapas. Para empezar, 1,000 fotografías fueron tomadas en el 2019 excepto por Guava_Good, en donde sólo 207 fueron tomadas en este año. Por el contrario, en el 2020 se agregaron entre 85 y 216 imágenes por clase. Por el contrario se tiene 793 imágenes de Guava_Good sin ninguna fecha, las cuales entran también dentro de los datos sin EXIF. De hecho si correlacionamos la figura 3.6.2 con la figura 3.6.1 se puede notar que la misma cantidad de imágenes tomadas en el 2020 fueron tomadas con el mismo teléfono (Realme 5 Pro).

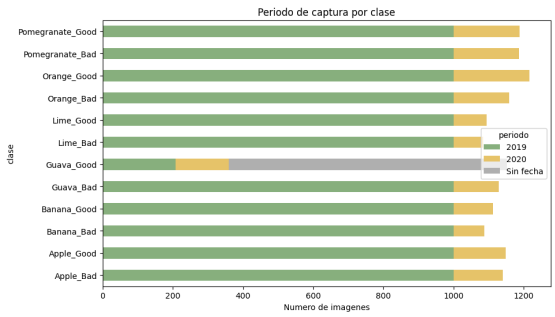


Figura 3.6.2 - Periodo de captura por clase

Por otro lado se analizó las observaciones vistas en la sección 3.4 acerca del fondo, debido a eso se analizó el borde exterior de cada imagen, en donde no aparece la fruta, en una muestra de 300 imágenes por clase. Es por eso que en cada borde se midió la proporción de píxeles con tonos verdes asociados a pasto u hojas y la proporción de píxeles claros asociados a los pisos blancos o de azulejos. Como se puede mostrar en la figura 3.6.3 se obtuvo un promedio de 7.9% del borde de las imágenes de buena calidad tiene tonos verdes, por el contrario se tiene un 1.6% en las fotos de mala calidad. Por otra parte, los tonos claros ocupan el 54.9% del borde en las imágenes de mala calidad y el 34.4% en las de buena calidad. Esto se puede notar más en Pomegranade_Good con un 23.4% de borde verde, ya que muchas de sus fotografías fueron tomadas sobre pasto. Sin embargo, cabe mencionar que el patrón no es el mismo para todas las frutas por ejemplo en las limas ocurre lo contrario ya que Lime_Good tiene más fondo claro (58.5%) que Lime_Bad (32.7%), es importante destacar que esto es solo una aproximación debido a que algunas imágenes el borde es ocupado por parte de la fruta o de la mano sin embargo, es suficiente concluir que el fondo varía de forma sistemática entre clases.

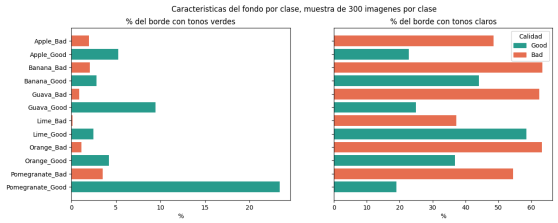


Figura 3.6.3 - Fondo por clase

Para finalizar cabe mencionar que ninguno de estos sesgos impide que se pueda utilizar el dataset, sino que obliga a interpretar los resultados con cuidado, debido a que una exactitud alta no garantiza que por sí sola el modelo haya aprendido a reconocer los defectos de la fruta. Para reducir estos riesgos se pueden plantear varias medidas como un aumento de los datos con cambios de brillo,

contraste, recorte y zoom y gracias a esto obligaría al modelo a no depender de los colores exactos de la cámara ni de la posición del fondo. Es por eso que durante la evaluación se medirá el desempeño de en el subconjunto del 2020, el cual como mencionado anteriormente fue tomado por un solo teléfono. Cabe mencionar, que se utilizara Grad-CAM para visualizar en qué regiones de la imagen se basa el modelo para decidir con el fin de comprobar si se observaba la fruta o el fondo y finalmente, se probará el modelo con imágenes de la carpeta mixta y con fotografías nuevas tomadas en condiciones distintas.

3.7 Hallazgos clave y Justificación

Gracias al análisis exploratorio permitió confirmar que después de la limpieza el dataset está adecuado para entrenar un modelo de clasificación y al mismo tiempo ayuda a identificar algunas características que influyen en el modelo.

Para empezar, el dataset se balanceo, ya que ahora las 12 clases presentan un rango entre 1,085 y 1,216 imágenes, si lo analizamos entre calidades se puede observar que existen 6,912 de buena calidad y 6,788 de mala calidad, por lo cual se puede observar que no hay mucha diferencia, por otro lado cada clase quedó repartida en 70% para entrenamiento, 15% para validación y 15% para pruebas, es por eso que la exactitud es una métrica representativa y no se requieren técnicas de compensación como pesos por clase. Para continuar, todas las imágenes están en el formato RGB con diferentes tamaños es por eso que se asignó una entrada de 224 x 224, para manejar un estándar entre todas las imágenes. Sin embargo, existen problemas como la baja resolución, y la deformación de imágenes no cuadradas.

Otro punto que cabe mencionar es que al realizar una revisión visual confirmó que las etiquetas eran consistentes, sin embargo se descubrieron dos tipos de problemas. El primer problema es la similitud entre limas y naranjas debido a que su aspecto físico es muy similar y el segundo es la confusión que puede haber en cuanto a las manchas de los plátanos maduros y las manchas de plátanos de mal estado.

Otro hallazgo, fue como dentro del análisis de las sesiones de captura mostró que de las 13,700 fotografías provienen de 588 sesiones, algunas de cientas de imágenes siendo casi idénticas. Es por eso, que era necesario dividir los datos por sesión para evitar los resultados inflados, no solo eso sino que se identificó que hay un gran probabilidad de que el modelo se sobreajuste con facilidad debido a que ve muchas veces escenas muy parecidas, por lo que es indispensable aplicar técnicas de regularización y aumento de datos a futuro.

Para finalizar, se identificaron algunos sesgos que podrían afectar el aprendizaje del modelo, siendo categorizados de la manera correcta, pero por diferentes características. Algunos ejemplos son: el teléfono con el que se capturó la foto, el tipo de fondo y el periodo de captura. Gracias a la identificación de esto se definirá las decisiones de las siguientes etapas, sin embargo lo pensado es realizar un aumento de datos en donde incluya cambio de brillo, contraste, recorte y zoom para reducir la dependencia de la cámara y el fondo. No solo eso, sino que evaluar las métricas por clase para verificar los errores anticipados, cabe mencionar que el desempeño será medido en los datos del 2020, debido a que son fotografías tomadas con el mismo teléfono y se utilizara Grad-CAM como extensión para verificar que parte de la imagen el modelo está basando sus decisiones en.

4 Modelo Base

4.1 Arquitectura

Este modelo base es una red neuronal convolucional sencilla entrenada desde 0, para

poder medir el efecto de cada mejora posterior. Esta versión no incluye ni aumentos de datos ni técnicas de regularización, justamente para observar el comportamiento de la arquitectura sin ayudas adicionales. En la siguiente tabla 4.1.1 se puede observar la arquitectura del modelo.

Capa	Configuración	Salida	Parámetros
Entrada	Imagen RGB	224 x 224 x 3	0
Rescaling	Divide cada píxel entre 225	224 x 224 x 3	0
Conv2D	32 filtros de 3 x 3,ReLu	222 x 222 x 32	896
MaxPooling2D	Ventana de 2 x 2	111 x 111 x 32	0
Conv2D	64 filtros de 3 x 3, ReLu	109 x 109 x 64	18,496
MaxPooling2D	Ventana de 2 x 2	54 x 54 x 64	0
Conv2D	128 filtros de 3 x 3, ReLu	52 x 52 x 128	73,856
MaxPooling2D	Ventana de 2 x 2	26 x 26 x 128	0
Flatten	Convierte el mapa a un vector	86,528	0
Dense	128 neuronas, ReLu	128	11,075,712
Dense(Salida)	12 neuronas, Softmax	12	1,548
Total			11,170,508

Figura 4.1.1 - Estructura del modelo

Gracias a la tabla 4.1.1 podemos observar la arquitectura que se implementó dentro del modelo en donde la primera capa, tiene un Rescaling(1./225) el cual como mencionado en las etapas anteriores es la normalización de las entradas así normalizando los valores de los pixeles del rango [0, 225] al rango [0,1]. Gracias a esto se estiliza el entrenamiento debido a que rechaza la entrada de los valores grandes, los cuales producen activaciones y gradientes demasiado grandes en las primeras capas. Cabe mencionar, que se incluyó como primera parte del modelo y no como un paso de preprocesamiento debido a que al hacer predicciones puede recibir directamente una imagen sin procesar y esto evita el error de olvidar normalizar.

Después se puede observar que siguen tres bloques convolucionales, los cuales cada uno de ellos contienen una capa de COV2D, la cual aplica los filtros de 3 x 3 pixeles los cuales recorren la imagen para detectar patrones locales, seguida de las capas de MaxPooling2D, las cuales reducen el tamaño del mapa a la mitad al conservar sólo el valor máximo de cada ventana de 2 x 2. Gracias a esta reducción disminuye el costo de cómputo y hace que cada filtro de cada siguiente capa abarque una región más amplia de la imagen original, también podemos observar como el número de filtros es duplicado por cada bloque empezando en 32 y terminando en 128, esto se debe a que las primeras capas detectan patrones simples como es el cambio de bordes, o color y a medida que va subiendo el valor de filtros los patrones se vuelven más complejos buscando texturas, formas, machas o combinando varias de estas. Es por eso que para estos patrones más complejos obviamente se necesitan más filtros para ser representados. Cabe mencionar, que todas las capas convolucionales utilizan la función de activación ReLU la cual está definida como $f(x) = máx(0,x)$, la cual introduce la no linealidad en la red con un costo de cómputo muy bajo.

Para finalizar, la capa de Flatten convierte el mapa de características de 26 x 26 x 128 en un vector de 86,528, los cuales posteriormente pasan a una capa densa de 128 neuronas con activación ReLu. Finalmente, la capa de salida tiene 12 neuronas, una por clase, con una activación de softmax la cual convierte las salidas de la red en probabilidades que suman 1, a continuación en la fórmula 4.1.2 se puede observar la fórmula de softmax, adecuada al modelo:

$$P(y = i \mid x) = \frac{e^{zi}}{\Sigma_{j=1}^{12} e^{zj}}$$

Fórmula 4.1.2 - Softmax adecuada al modelo

En donde: Zi es la salida de la neurona correspondiente a la clase i, o sea la clase con mayor probabilidad es la predicción del modelo.

Cabe mencionar, que un aspecto importante de la arquitectura es la distribución de parámetros, en donde de los 11.17 millones de parámetros del modelo, 11.08 millones alrededor del 99.2%, están en la conexión entre la capa flatten y la capa densa debido a que uno de los valores del vector se conecta con cada una de las 128 neuronas. La concentración de parámetros en una sola capa del modelo ayuda a que se pueda aprender los datos de entrenamiento de manera muy fácil, por lo que es propenso a sobre ajustarse.

4.2 Función de costo

La función de costo mide qué tan lejos están las predicciones del modelo de las etiquetas reales y es la cantidad que el algoritmo busca minimizar durante el entrenamiento, debido a que el problema es de clasificación multiclase con una salida de softmax, se tomó la decisión de utilizar la entropía cruzada categórica, definida por un lote de N imágenes

$$L = -\frac{1}{N} \Sigma_{n=1}^{N} \Sigma_{i=1}^{12} y_{n,i} log(\widehat{y_{n,i}})$$

Fórmula 4.2.1 - Cross Entropy adecuada al modelo

En donde: yn,i es la etiqueta real en codificación one-hot, en otras palabras escribe un 1 si la imagen n pertenece a la clase i y 0 en el caso contrario. Por otro lado, $\widehat{y_{n,i}}$,es la probabilidad de que el modelo asigna a la clase i. Como el vector one-hot es solo la clase correcta vale 1, la suma se reduce a un solo término por imagen y así obteniendo $-\ log(\widehat{y_{correcta}})$. En otras palabras es decir, la función de costo solo depende de la probabilidad que el modelo asigna a la clase correcta.

Para entender mejor la fórmula se realizó un ejemplo, si tenemos una imagen de la clase de Lime_Bad y el modelo le asigna la probabilidad de 0.9 a esa clase, el costo en realidad es de $-\ log(0.9)$, lo cual da un resultado de 0.105. Por otro lado si asigna una probabilidad de 0.1, el costo subiría a $-\ log(0.1) = 2.303$ y en el caso de que le asigna un 0.01 alcanzaría un valor de 4.605, es por eso que podemos notar como la función crece rápidamente a medida que la probabilidad de la clase correcta se acerca a 0. Como se puede notar en estos ejemplos el modelo penaliza a las predicciones incorrectas hechas con alta confianza que a las predicciones dudosas.

Esta función se eligió en lugar del MSE debido a que interpreta las salidas del modelo como probabilidades, lo cual es coherente con la capa de softmax y con el objetivo de clasificar. Por otro lado, al combinarla con softmax el gradiente respecto a cada salida de la red se simplifica a $\widehat{y_i} - y_i$, la diferencia entre probabilidad predicha y real. Básicamente este gradiente es proporcional al error siendo muy grande

cuando el modelo se equivoca mucho y pequeño cuando está cercas de la respuesta correcta. Por el contrario, el MSE se vuelve muy pequeño cuando la salida de softmax está saturada, aún si la predicción es incorrecta haciendo el aprendizaje lento.

Cabe mencionar, que un modelo que le asigna la misma probabilidad a las 12 clases tiene un costo de -log(1/12) = 2.485, lo cual es un valor esperado al inicio del entrenamiento, dentro del modelo base cabe mencionar que el valor de pérdida bajo hasta 0.006 en la última época, lo cual menciona que el modelo asignó una probabilidad de 1 a la clase correcta en prácticamente todas las imágenes de entrenamiento. Por otro lado, la pérdida de validación subió hasta 3.754 por encima que incluso un valor de un modelo que adivina al azar. Esto no significa que el modelo sea peor que un modelo al azar debido a que dentro de validación obtuvo un 60.5% de exactitud, sino que el modelo está haciendo predicciones con mucha confianza y la función de costo la penaliza por ello, lo cual nos da señal de que el modelo tenga sobreajuste.

4.3 Optimizador e hiperparametros

Por otro lado, tenemos el optimizador el cual es el algoritmo que ajusta los pesos de la red para minimizar la función de costo. Cabe mencionar, que la base de los optimizadores utilizados en las redes neuronales comúnmente es el descenso de gradiente, en donde en cada paso se calcula el gradiente de la función de costo respecto a cada peso, el cual indica la dirección que aumenta el error, y el peso, el cual mueve una pequeña de cantidad en la dirección contraria. En la fórmula 4.3.1 se puede observar esta fórmula:

$$w_{t+1} = w_t - \eta \nabla L(w_t)$$

Fórmula 4.3.1 - Descenso de gradiente

En donde: wt es el valor del peso en el paso t, por otra parte $\nabla L(w_t)$es el gradiente de la función de costo y η Es la tasa de aprendizaje, la cual es la que controla el tamaño de cada paso. Es importante destacar que, normalmente el gradiente no se calcula con todo el conjunto de entrenamiento, sino con un lote pequeño de imágenes, lo que se conoce como descenso de gradiente estocástico por mini-lotes. Esto hace que cada actualización sea rápida aunque el gradiente de cada lote sea solo una aproximación ruidosa del gradiente real.

Sin embargo, para este trabajo se utilizó el optimizador ADAM o también conocido como Adaptive Moment Estimation, el cual es una variante del descenso de gradiente que mejora dos de sus límites. Para empezar, se utilizó debido a que Adam mantiene un promedio móvil de los gradientes recientes que funciona como una inercia la cual suaviza el ruido entre lotes y acelera el avance de direcciones consistentes. Por otro lado, mantiene un promedio móvil de los gradientes al cuadrado, que utiliza para ajustar la tasa de aprendizaje de forma individual para cada peso , lo que significa que los pesos con gradientes grandes dan pasos más pequeños y los pesos con gradientes pequeños dan pasos más grandes. A continuación se muestra la fórmula de regla de actualización de pesos de Adam:

$$w_{t+1} = w_t - \eta \frac{\widehat{m_t}}{\sqrt{\widehat{v_t}} + \epsilon}$$

Fórmula 4.3.2 - Actualización de pesos de Adam

En donde, $\widehat{mt}$ y $\widehat{vt}$ son los momentos corregidos para compensar que ambos promedios empiezan en 0, y ϵ es la constante que evita que se divida entre 0. En este caso se eligió Adam debido a que converge rápidamente sin ajustar curiosamente la tasa de aprendizaje y porque sus valores defecto funcionan bien con la mayoría de problemas de visión, lo cual lo hace adecuado para el modelo que se está construyendo.

En la tabla 4.3.3, se puede observar los hiperparametros utilizados:

Hiperparametros	Valores	Justificación
Tasa de aprendizaje	0.001	Valor por defecto de Adam
B1,B2, e	0.9, 0.999, 10-7	Valores por defecto de Adam en Keras
Tamaño de lote	32	Equilibrio entre estabilidad del gradiente y uso de memoria
Epochs	15	Suficientes para observar el comportamiento en entrenamiento
Early Stopping	No	Para observar si existe sobre ajuste
Seed	42	Reproducibilidad del entrenamiento

Tabla 4.3.3 - Hiperparametros

En este caso con 9,592 imágenes de entrenamiento y lotes de 32, cada epoch consta de 300 actualizaciones de pesos, por lo que nos da un total de 4,500 actualización en las 15 epochs. Cabe mencionar que para el modelo base se mantuvieron los valores por defecto del optimizador y no se utilizó el early stopping, para observar el comportamiento natural de la arquitectura durante todo el entrenamiento, sin ninguna técnica de mejora para tener una referencia clara contra el modelo en donde se amplían las mejoras.

4.4 Entrenamiento

Como mencionado anteriormente, el modelo base fue entrenado con 15 epochs, en la figura 4.4.1 se puede observar la evolución de la función de costo y la exactitud que el modelo obtuvo en los grupos de entrenamiento y validación.

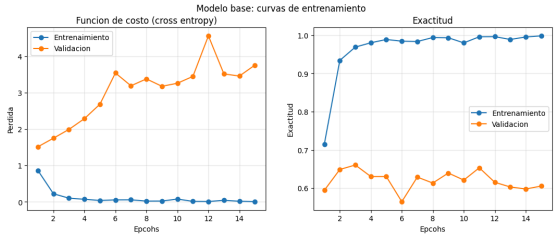


Figura 4.4.1 - Evolución de la función de costo y de la exactitud en los grupos de entrenamiento y validación

Como se puede observar en la figura 4.4.1 se puede notar como el conjunto de entrenamiento aprendio muy rapido, esto se puede notar como al terminar su primera epoch obtuvo una exactitud de 0.71 o 71.5%, por otra parte en la segunda epoch se obtuvo un valor de ~93% y a partir de la 5 pechos se mantuvo de cierta manera constante debido a que no llegó a bajar de ~98%, y para su última epoch se obtuvo un valor del ~99.8% y es por eso que la perdida de entrenamiento en la figura de la función de costo va bajando de forma continua de 0.861 a 0.006, es por eso que podemos concluir que el modelo clasifica casi todas las imágenes de la manera correcta y con una confianza casi del 100%.

Por otra parte, se tiene el conjunto de validación el cual es notoriamente diferente. Para empezar, con la pérdida empezó con un valor de 1.513 y a partir de ahí se puede observar cómo fue aumentando hasta llegar entre 3.538 y 4.557, lo cual es más del doble que su mínimo. Por otra parte, en la figura de exactitud podemos ver como empezó con un 59.4% y de ahi mejoro hasta la epoch 3, en donde empezó a deteriorarse llegando a su punto mínimo siendo del 56.5%, a medida que va continuando las epochs, se puede notar que el modelo deja de mejorar entre 56% y 65%, así terminando con un valor de 60.5%. Como podemos notar el modelo empezó a empeorar después de la tercera epochs, en donde es notorio que su desempeño se estancó y su pérdida empeoró desde la segunda epoch.

Como se puede notar es una gran separación entre ambas curvas, lo cual es una obvia señal de que existe sobreajuste dentro

del modelo. Como se mencionó en las secciones anteriores el modelo dejó de aprender los patrones generales y se empezó a memorizar los detalles específicos de las imágenes de entrenamiento. Es por eso que al terminar el entrenamiento, la brecha que existe entre la exactitud de entrenamiento y de validación fue de ~38 pps. Por otra parte, cabe mencionar que se observan dos picos en la perdida de entrenamiento en específico en las epochs 6 y 10, las cuales indican cierta inestabilidad en el entrenamiento, lo cual es comprensible dentro de un modelo que cuenta con 11 millones de parámetros en una sola capa densa sin ningún tipo de regularización.

Como en este caso no se está utilizando la técnica del early stopping, los pesos guardados corresponden a la de la última epoch en vez de a la de mejor desempeño en validación. Lo cual es una técnica considerada para la sección de mejoras, debido a que conserva los pesos de la mejor epoch, y para el proceso de entrenamiento cuando la pérdida de validación deja de mejorar así evitando el sobreajuste.

4.5 Evaluación en validacion y diagnóstico

Cabe mencionar que el modelo base se evaluó con el conjunto de evaluación, el cual está formado por 2,055 imágenes de sesiones que el modelo no vio durante su fase de entrenamiento. Es importante destacar que el grupo de pruebas no fue utilizado en esta etapa debido a que va a ser reservada para el modelo final de modo que las decisiones tomadas durante las mejoras no estén influenciadas por él. Por lo analizado el modelo obtuvo una exactitud del 80.5% y una pérdida de 3.754 dentro del grupo de validación. Sin embargo sigue estando muy lejos de lo observado en el grupo de entrenamiento siendo de 99.8%.

Dentro de la tabla 4.5.1 se puede observar la precisión, exhaustividad y el F1-score de cada clase. Lo que indica la precisión es qué proporción de las imágenes asignó el modelo a una clase en donde sí pertenece. Por otro lado, la exhaustividad indica cuáles imágenes fueron identificadas correctamente y para finalizar el F1-score es la medida armónica de ambas.

Clase	Precisión	Exhaustividad	F1-score
Apple_Bad	0.644	0.550	0.593
Apple_Good	0.480	0.751	0.586
Banana_Bad	0.764	0.515	0.615
Banana_Good	0.796	0.681	0.734
Guava_Bad	0.607	0.639	0.622
Guava_Good	0.517	0.705	0.597
Lime_Bad	0.550	0.577	0.563
Lime_Good	0.889	0.393	0.545
Orange_Bad	0.547	0.400	0.462
Orange_Good	0.443	0.514	0.476
Pomegranate_Bad	0.723	0.601	0.656
Pomegranate_Good	0.680	0.921	0.783
Promedio	**0.637**	**0.604**	**0.603**

Tabla 4.5.1 - Precisión, Exhaustividad y F1-Score

Como se puede observar en la tabla 4.5.1 se puede observar la irregularidad entre clases, en donde Pomegranate_Good tiene una exhaustividad más alta siendo de 92.1%, sin

embargo, contiene una precisión de 68.0%, lo que significa que el modelo asigna esta clase con demasiada frecuencia. Por el contrario, la clase de Lime_Good cuando el modelo predice esta clase acierta el 88.9% de las veces, sin embargo solo reconoce el 39.3% de las limas buenas. Para finalizar se tiene las clases de Orange_Good y Orange_Bad siendo las de peor desempeño, debido a que cuentan con un F1-Score de 0.462 en el caso de Orange_Bad y 0.476 en el caso de Orange_Good. Posteriormente se presentará la figura 4.5.2 en donde se muestra la matriz de confusión:

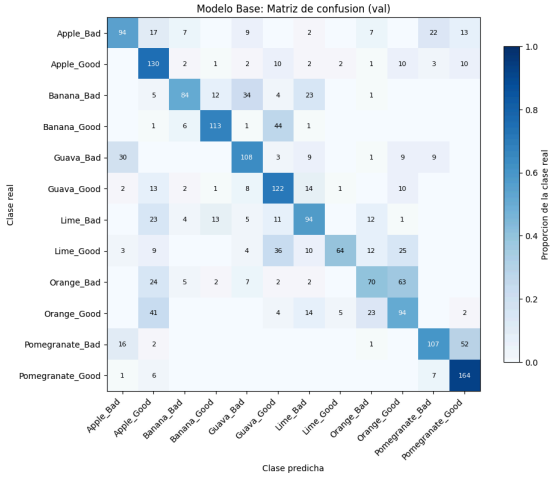


Figura 4.5.2 - Matriz de confusión (val)

Como se puede analizar en la figura 4.5.2 la confusión más frecuente ocurre entre las naranjas malas y las buenas, y las limas de buena calidad son confundidas con las naranjas de buena calidad, como se esperaba debido a que son frutas parecidas. Sin embargo, podemos notar como de los 811 errores del modelo, 610, o sea el 75.2% corresponde a la fruta equivocada y solo 201 o el 24.8% es debido a que identifica la fruta correctamente, pero la categoriza en la calidad equivocada. De hecho, es lo contrario a lo esperado debido a que es más fácil distinguir entre las clases de frutas que del estado de calidad.

Otro caso que cabe mencionar, es como de las 6 de las 8 confusiones más frecuentes el modelo se equivoca de fruta, pero conserva la calidad, en otras palabras confunde el plátano bueno con una guayaba buena o una guayaba mala con una manzana mala. Por lo que se puede concluir que el modelo no aprendió a reconocer la forma de cada fruta, sino las características asociadas a la calidad. Como se mostró y redactó en la sección 3.6, el fondo y el teléfono de captura están relacionados con la calidad, por lo que existe una probabilidad de que el modelo haya aprendido a reconocer esas características, por ejemplo como se mencionó en esa sección el piso claro de las fotografías estaba asociado con las frutas de mala calidad, por otro lado los fondos variados estaban asociados a las fotos de buena calidad.b Esta hipótesis se verificará utilizando la herramienta de Grad-Cam, dentro de la evaluación del modelo final.

Para finalizar, tenemos el diagnóstico en donde podemos concluir que el modelo base presenta un serio caso de sobreajuste, en otras palabras significa que es un problema de alta varianza y no de falta de capacidad, debido a que es capaz de aprender de manera perfecta el conjunto de entrenamiento, pero le cuesta problemas las imágenes nuevas. Gracias a esto se identificaron tres causas principales, para iniciar, la capa densa concentra el 99.2% de los 11.7 millones de parámetros del modelo, por lo que genera una gran capacidad de memorización. Para continuar, el dataset contiene muchas imágenes con las mismas escenas debido a que las 9,592 imágenes de entrenamiento provienen de sesiones casi idénticas y es por eso que el modelo ve muchas veces la misma fruta, con el mismo fondo e iluminación. Para finalizar, el modelo no tiene ninguna técnica que lo obligue a generalizar, debido a que no hay aumento de datos, regularización ni early stopping. Es debido a estas tres causas que se definen las mejores del modelo de la siguiente sección.

5 Mejoras al modelo

5.1 Estrategia de mejora

Dentro de la sección 4.5, la cual habla de la evaluación del modelo base se identificó que existía un sobreajuste severo con tres causas principales, la concentración del 99.2% de los parámetros en la capa densa, la poca variedad de escenas dentro del dataset, y finalmente, la ausencia de técnicas que ayuden al modelo a generalizar. Es por eso que para corregir esto se diseñaron 5 experimentos incrementales en donde cada uno parte del anterior y modifica sólo un aspecto para que se nota la diferencia en el desempeño que dicho atributo haya cambiado, dentro de la tabla 5.1.1 se pueden observar los experimentos realizados:

Exp.	Cambio respecto al anterior	Causa que ataca
E1	Global Average Pooling 2D en lugar de Flatten	Exceso de parámetros en la capa densa
E2	Batch Normalization y Dropout	Memorización e inestabilidad
E3	Aumento de datos y early stopping	Poca variedad de escenas
E4	Transfer Learning con MobileNetV2 congelada	Pocos datos para aprender características desde 0
E5	Fine Tunning de MobileNet V2	Adaptar las características pertenecientes a las frutas

Tabla 5.1.1 - Experimentos realizados

En este caso, todos los experimentos realizados fueron evaluados con el conjunto de validación y con las mismas métricas de exactitud, pérdida, F1_score, la brecha de exactitud entre entrenamiento y validación, y la exactitud del subconjunto del 2020. Cabe mencionar, que la exactitud del subconjunto del 2020 es marcado como un parámetro de evaluación debido a que este está formado por 359 imagenes de validación tomadas con el mismo teléfono y esto sirve para verificar que tanto el modelo depende de las condiciones de captura, algo que fue mencionado en la sección 3.6. Para finalizar, el conjunto de prueba no fue utilizado en esta sección, debido a que está siendo reservado para el modelo final.

5.2 E1: GlobalAveragePooling2D

Como fue observado en la sección 4.1, el modelo base utilizaba la capa de Flatten la cual convertía el mapa de características de 26 x 26 x 128 en un vector de 86,528 valores, cada uno conectado a una capa densa, debido a que existía un exceso de parámetros en la capa densa se decidió sustituir a flatten por un Global Average Pooling, debido a que este calcula el promedio de cada uno de los 128 mapas de características y produce un vector de solo 128 valores, entonces en lugar de tener 11,170,508 paso a 111,308 siendo una reducción del 99%.

No solo ayuda a reducir la necesidad de memorización, sino que también modifica lo que aprende el modelo. En el caso de utilizar flatten, cada peso de la capa densa estaba asociado a una posición específica de la imagen, por lo que el modelo puede llegar a aprender posiciones de las imágenes. Por ejemplo, podría aprender que existe pasto verde en la esquina superior izquierda, lo cual con eso se guiaría y la categoriza dentro de alguna fruta de buena calidad, debido a que como mencionado en las secciones anteriores los píxeles verdes están asociados normalmente a las frutas de buena calidad. Por el contrario, con el Global Average Pooling, el modelo solo sabe si esta una característica presente en la imagen y con qué intensidad, pero no en donde, por lo que lo obliga a basarse en que hay en la imagen y no en la posición exacta de cada elemento. Cabe mencionar que esto no elimina el sesgo del fondo, sino que reduce sus capacidades de memorizar los detalles ligados a las posiciones específicas.

Por lo que se observó el resultado fue de las mejoras más grandes de los experimentos, debido a que la exactitud de validación subió de 60.5% a 74.4%, lo cual es un aumento de 13.9 puntos, por otro lado la pérdida de validación bajo de 3.754 a 1.623 y la brecha de entrenamiento y validación se redujo de 39.3 a 18.4. Por otro lado la exactitud del entrenamiento bajó de 99.8% a 92.8%, lo cual confirma que el modelo memoriza menos, sin embargo el sobreajuste no desapareció por completo debido a que la pérdida de validación alcanzó su mínimo en la epoch 4 y posteriormente empezó a volver aumentar. Para finalizar, se presentará la figura 5.2.1, en donde se puede observar el modelo con este experimento:

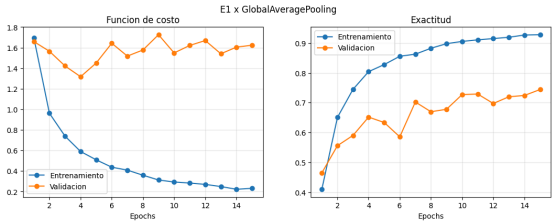


Figura 5.2.1 - Experimento 1 x Global Average Pooling

5.3 E2: BatchNormalization y DropOut

Por otro lado en el experimento 2 se agregaron dos técnicas de regularización a la arquitectura del experimento 1, después de cada capa convolucional se añadió una capa de Bach Normalization, la cual básicamente normaliza las activaciones de casa lote para que tengan una media de cero y varianza de uno, esto con el objetivo de estabilizar y acelerar el entrenamiento. Cabe mencionar que también antes de la capa de la salida se añadió un DropOut de 0.5, el cual en cada caso de entrenamiento desactiva aleatoriamente la mitad de las neuronas de la capa densa con el objetivo de que el modelo no dependa de ciertas combinaciones específicas de neuronas.

Lamentablemente, el experimento 2 fue marcado como fallido, debido a que tuvo un peor desempeño, como se observó la exactitud de la validación bajo al 60.8%, y la brecha subió 31.0 puntos. Sin embargo, lo más notable del experimento fue la inestabilidad del entrenamiento, mientras que la exactitud del entrenamiento subió de forma continua, la exactitud de validación varió de manera errática entre el 10.4% y el 65.1%, de una epoch a otra. Esto puede ser observado en la figura 5.3.1:

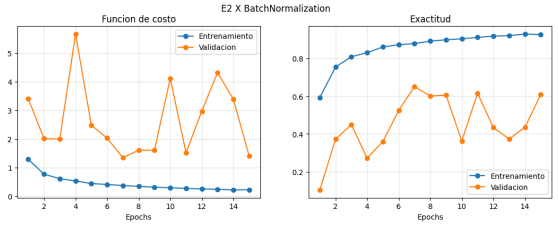


Figura 5.3.1 - Experimento 2 x Batch Normalization y DropOut

La inestabilidad de este experimento puede ser explicada por el funcionamiento del Batch Normalization debido a que durante el entrenamiento, cada capa normaliza las activaciones con la media y la varianza del lote actual, por lo que durante la evaluación, usa promedios móviles de esas estadísticas, acumulados a lo largo del entrenamiento. Cuando el modelo se entrena desde 0 con una tasa de aprendizaje alta (0.001), los pesos cambian rápidamente y los promedios móviles quedan rezagados respecto a las estadísticas reales. Es por eso que el modelo que se evalúa en validación no se comporta igual al que se entrena y este resultado muestra que este efecto de regularización falló en mi caso.

5.4 E3: Aumento de datos y early stopping

Por otro lado en el experimento 3 se agrego un aumento de datos a la arquitectura del experimento 2, por lo que antes de entrar a la red cada imagen de entrenamiento se transforma al azar en cada epoch, ya sea que se voltee horizontalmente, se rote hasta 54º, se aplique un zoom de hasta el 20%, se desplace hasta un 10%, se modifique su brillo, o su contraste hasta de un 20%. Gracias a estas transformaciones se generan imágenes nuevas

a partir de las existentes y atacan dos de los problemas vistos, la del dataset con poca variedad de escenas y la de los sesgos de captura identificados en la sección 3.6. Cabe mencionar que el aumento de datos solo se aplica durante el entrenamiento, en validación y pruebas las imágenes no se modifican, porque se necesita evaluar el modelo con imágenes reales. Debido a que cada epoch es distinto se permitieron hasta 40 epochs con early stopping, el cual detiene el entrenamiento si la pérdida de validación no mejora durante 5 epochs consecutivas y restaura los pesos de la mejor epoch.

Lamentablemente, el experimento 3 obtuvo el peor resultado de todos los experimentos, siendo de 48.6% de exactitud en validación. Sin embargo cabe destacar que este resultado no significa que el aumento de datos sea perjudicial, sino que no se pudo evaluar correctamente en esta arquitectura. Otro caso es que la inestabilidad del Batch Normalization del experimento 2 se agravó debido a que los cambios de brillo y contraste aumentaron aún más la diferencia entre las estadísticas de los lotes de entrenamiento y las imágenes sin modificar de validación. Por otro lado, la pérdida de validación cambió demasiado ya que el early stopping reaccionó al ruido y detuvo el entrenamiento en la epoch 7. En otras palabras, el modelo final del experimento 3 está sub entrenado y no sobre ajustado, debido a la interrupción del entrenamiento. Esto se puede observar en la figura 5.4.1:

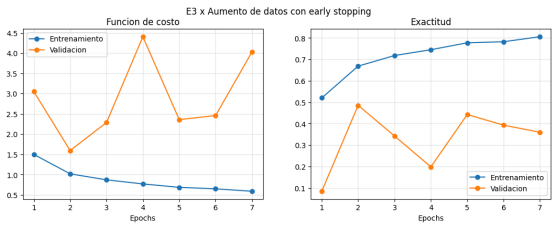


Figura 5.4.1 - Experimento 3 x aumento de datos con early stopping

5.5 E4:Transfer learning con MobileNetV2

Gracias a los experimentos anteriores, se pudo notar que con unas 9,600 imágenes provenientes de pocas escenas distintas es difícil que una red neuronal aprenda desde cero las características visuales. Es debido a esto que el experimento 4 tiene el propósito de utilizar una red convolucional pre entrenada justo para que en lugar de aprender esas características se utilice ImageNet el cual contiene un conjunto de 1.2 millones de imágenes de 1,000 categorías, las cuales ya están entrenadas a detectar bordes, texturas, formas, y objetos, lo cual es útil para casi cualquier problema de visión. En este caso se eligió MobileNetV2 debido a que es una arquitectura ligera, ya que cuenta con 2.26 millones de parámetros que se entrenan rápidamente en Colab y es adecuada para el problema de la inspección de calidad.

Cabe mencionar que la base de MobileNetV2 se congeló de modo que sus pesos no cambiarán durante el entrenamiento y sobre ella se agregó una nueva cabeza de clasificación utilizando Global Averga Pooling 2D, Dropout de 0.3 y una capa densa de neuronas con softmax. Así de los 2,273,356 parámetros del modelo solo se entrenaron 15,372. Debido a que MobileNetV2 fue entrenado con píxeles de rango de -1 a 1 y la normalización cambió a un Rescaling. Por otro lado, se conservaron el aumento de datos y el early stopping del tercer experimento.

De hecho el resultado de este experimento fue de los saltos más grandes de todo el proceso debido a que la exactitud de validación subió a 85.9% subiendo 25.4 puntos con respecto al modelo base, por otro lado la pérdida bajó a 0.501. Como se puede observar en la figura 5.5.1, se puede observar como desde la primera epoch el modelo alcanza un 84% en validación, lo que muestra que las características de ImageNet fueron útiles para distinguir las frutas. También se puede

observar como la brecha entre entrenamiento y validación se redujo a 8.2 puntos y las curvas de validación fueron más estables, esto se debe a que el modelo tiene poca capacidad de memorizar, por otro lado, las capas de batch normalization de la base congelada funcionan siempre en modo de inferencia con estadísticas fijas aprendidas de ImageNet, por lo que no se presentó el problema que fue visto en el experimento 2 y 3. Gracias a estas condiciones es que el aumento de datos funcionó de la manera esperada.

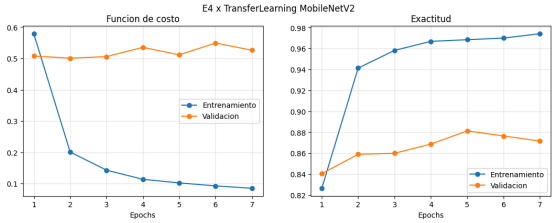


Figura 5.5.1 - Experimento 4 MobileNetV2

5.6 E5: Fine-tunning de MobileNetV2

Cabe mencionar que las características de ImageNet son generales, es por eso que no están especializadas en detalles como son las manchas, el moho, las arrugas de la fruta. Es por eso que el propósito del experimento 5 fue aplicar un fine-tuning partiendo los pesos del experimento 4 y descongelando las primeras capas de MobileNetV2 excepto las primeras 30, las cuales detectan los patrones muy básicos como los bordes, colores, y son útiles para cualquier imagen y posteriormente se continuó el entrenamiento. Gracias a esto, el número de parámetros entrenables pasó de 15,372 a 2,219,708. Para no destruir las características pre entrenadas con actualizaciones demasiado grandes la tasa de aprendizaje se redujo 100 veces y la paciencia del early stopping se fijó en 4 epochs.

Gracias a estos cambios podemos concluir que el experimento 5 es el mejor modelo obtenido debido a que alcanzó una exactitud de validación de 90.5%, una pérdida de 0.365 y un F1_score de 0.905, y como es observado en la figura 5.6.1 la mejor epoch fue de 7 a 11, sin embargo a pesar de tener muchos más parámetros entrenables que en el experimento 4, la brecha de entrenamiento y validación se mantuvo en 8.0 puntos, gracias a la tasa de aprendizaje baja, el aumento de datos y el early stopping. Cabe destacar que también el número de errores bajo de 811 en el modelo base a 195. De hecho los errores que se presentaron del modelo es que clasificó bien a la fruta, sin embargo se equivocó en la calidad de ella, sin embargo aun así bajaron de 201 a 26, lo que muestra que el fine-tunning le permitió aprender los detalles de la superficie de la fruta y distinguir de una fruta buena de una mala.

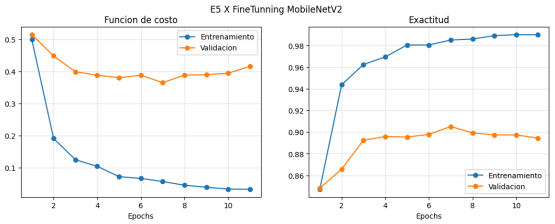


Figura 5.6.1 - Experimento 5 MobileNetV2 con Fine-Tuning

5.7 Comparación

Para finalizar, se muestra la tabla 5.7.1 en donde se resumen todos los resultados de los experimentos en el conjunto de validación:

Experimento	Parámetros entrenables	Epochs	Exactitud (entr.)	Exactitud (val.)	Brecha	Pérdida (val.)	F1 macro	Exactitud 2020
E0 · Modelo base	11,170,508	15	99.8 %	60.5 %	39.3	3.754	0.603	25.1 %
E1 · GlobalAvgPooling	111,308	15	92.8 %	74.4 %	18.4	1.623	0.747	32.3 %
E2 · + BatchNorm y Dropout	111,532	15	92.7 %	60.8 %	31.9	1.414	0.618	24.2 %
E3 · + Aumento de datos	111,532	7	66.9 %	48.6 %	18.3	1.596	0.475	20.1 %
E4 · MobileNetV2 congelada	15,372	7	94.1 %	85.9 %	8.2	0.501	0.859	51.8 %
E5 · Fine-tuning	**2,219,708**	**11**	**98.5 %**	**90.5 %**	**8.0**	**0.365**	**0.905**	**64.9 %**

Tabla 5.7.1 - Tabla de comparación entre experimentos

Gracias a los experimentos hubieron algunos cambios que funcionaron, en este caso fueron 3, los cambios en donde se noto un mejoró del modelo. El primero fue sustituir a flatten por Global Average Pooling 2D aumentando 13.9 puntos y elimino el 99% de los parámetros que eran la principal fuente de moralización así obligando al modelo a basarse en las características en vez de la posición. Por otro lado, fue el transfer learning

en donde aumentó 25.4 puntos con respecto al modelo base y resolvió el problema de la falta de escenas para que el modelo aprendiera las características visuales desde 0, reutilizando lo aprendido con 1.2 millones de imágenes. Para finalizar, el tercero fue el fine-tunning mejorando 4.6 puntos con respecto al experimento 4, en este se adoptó esas características generales a los detalles específicos de la superficie de las frutas, lo cual redujo los errores de la categorización de imágenes.

Por el contrario también hubo cambios los cuales no funcionaron a lo esperado, para empezar el Batch Normalization entrenada desde cero volvió inestable la evaluación del modelo, y el aumento de datos lo agravó, al grado que el early stopping para el entrenamiento antes de tiempo. Es por eso que se puede concluir que una técnica de regularización no mejora el modelo por sí sola, sino que depende de su interacción con la arquitectura e incluso con el entrenamiento.

Cabe mencionar que existen algunas limitaciones del modelo, aunque el experimento 5 haya alcanzado un 90.5% de exactitud en la etapa de validación, su desempeño dentro del conjunto de los datos del 2020 fue de solo 64.9%. De hecho todos los modelos dependen de esto como se puede observar en la tabla 5.7.1 se puede observar como todos los modelos les va peor en esta subcategoria, lo cual confirma lo que se habló dentro de la sección del análisis exploratorio, en la cual una parte del desempeño del modelo depende de las condiciones de captura y no solo de la fruta. Gracias al transfer learning y al fine-tuning se pudo reducir esto, sin embargo es notorio que no fue eliminado. Dentro de la figura 5.7.2 se puede observar la matriz de correlación del experimento 5, y gracias a ella se puede notar que la clase de la naranja sigue siendo de las más difíciles, ya que es en la cual comete más errores, estos aspectos serán analizados a más a detalle en la evaluación final.

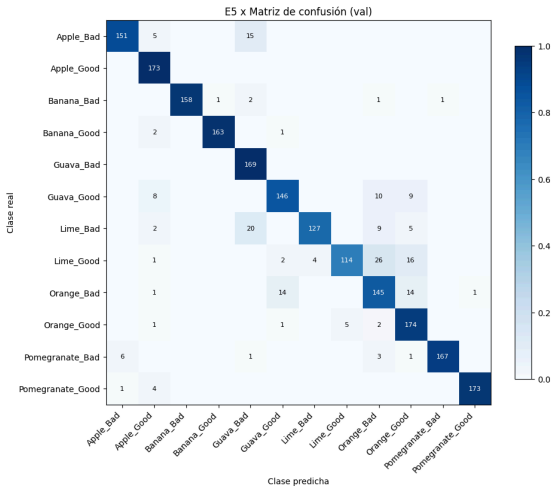


Figura 5.7.2 - Matriz de correlación, experimento 5

6 Evaluación final en test

6.1 Desempeño en el conjunto de prueba

El modelo final, se evaluó una sola vez con el conjunto de prueba, el cual está formado por 2,053 imágenes de sesiones de captura, las cuales no fueron utilizadas ni para entrenar ni para tomar ninguna decisión durante el desarrollo. Como se puede observar en la tabla 6.1.1 este conjunto alcanzó una exactitud de 83.3% con una pérdida de 0.631 y un F1-Score 0.834.

Conjunto	Exactitud	Pérdida	F1_Score
Validación	90.5%	0.365	0.905
Pruebas	83.3%	0.636	0.834

Tabla 6.1.1 - Desempeño final

Como se pudo observar en la tabla 6.1.1 el desempeño en prueba es de 7.2 puntos menor que en validación, esta diferencia es notable por dos cosas. Para empezar, el conjunto de validación se utilizó en 5 experimentos para elegir el mejor modelo y decidir en cual epoch se tenía que detener el entrenamiento, por lo que el resultado de validación está ligeramente sesgado a favor del modelo elegido. Por otro lado, el conjunto de prueba ofrece una estimación independiente y

más realista en el desempeño con imágenes nuevas. La otra razón, es debido a que el dataset contiene pocas sesiones por clase, cada conjunto está formado por relativamente pocas escenas distintas y una sola sesión difícil puede concentrar muchos errores como se observa en la sección 6.2. Sin embargo, el modelo final supera de manera exponencial al modelo base cuya exactitud de validación fue de 60.5%. A continuación en la tabla 6.1.2, se mostrará la precisión, exhaustividad y el F1_score por clase.

Clase	Precisión	Exhaustividad	F1-score
Apple_Bad	0.967	0.524	0.679
Apple_Good	0.669	0.977	0.794
Banana_Bad	1.000	1.000	1.000
Banana_Good	1.000	1.000	1.000
Guava_Bad	0.935	0.841	0.885
Guava_Good	0.953	0.711	0.815
Lime_Bad	0.781	0.767	0.774
Lime_Good	0.905	0.933	0.918
Orange_Bad	0.494	0.457	0.474
Orange_Good	0.587	0.819	0.683
Pomegranate_Bad	1.000	0.989	0.994
Pomegranate_Good	0.983	0.989	0.986

Tabla 6.1.2 - Precisión, Exhaustividad, y F1-Score del modelo final

Como se puede observar en la tabla 6.1.2 el desempeño varía demasiado por fruta, por ejemplo el plátano y la granda muestran una clasificación perfecta con un F1 score entre 0.986 a 1.000 en sus cuatro clases. Por el contrario, la naranja se puede observar que es la fruta más difícil ya que en la clase de Orange_Bad tiene un F1 score de 0.474 y Orange_Good tiene de 0.683. Para finalizar podemos notar un caso curioso en las manzanas en donde el modelo reconoce casi todas las manzanas buenas con un 97.7% de exhaustividad, pero solo la mitad de las malas con un 52.4%, debido a que muchas de las manzanas de mal calidad son clasificadas dentro de las de buen estado.

6.2 Análisis de Errores

El modelo final cometió 342 errores en las 2,053 imágenes de prueba, en la figura 6.2.1 se puede mostrar la matriz de confusión:

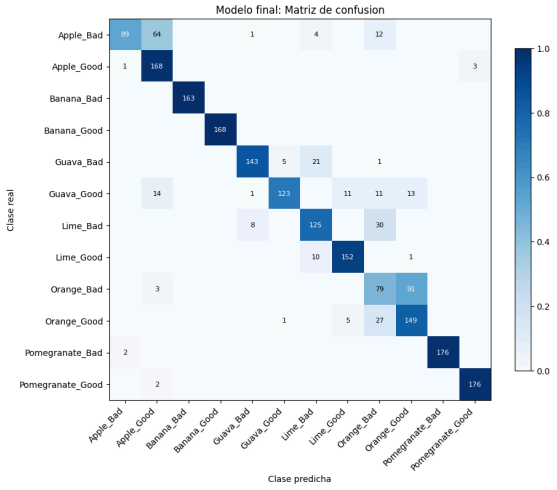


Figura 6.2.1 - Matriz de confusión del modelo final

Como se puede observar en la figura 6.2.1, los errores más notados son las naranjas malas clasificadas como buenas y las manzanas malas clasificadas como buenas, estos dos casos suman el 155 de los errores, formando el 45% total de todos los errores.Esto se relaciona con la naturaleza de los defectos debido a que algunas naranjas y manzanas de mala calidad tienen defectos leves como la piel ligeramente arrugada o

pequeñas manchas, y teniendo una resolución de 224 x 224 píxeles lo hace más difícil distinguir de una fruta de buena calidad de una de mala. Por otro lado, al revisar los errores con mayopr confianza se encontró que tres de los seis pertenecen a la misma sesión de captura, o sea que es la misma naranja sostenida en la mesa de madera fotografiada con ángulos similares. Lo cual confirma que cuando una sesión le resulta al modelo difícil de clasificar hay una gran probabilidad de que todo el conjunto de sesiones falle.

Cabe mencionar, que un cambio en contra del modelo base es el tipo de errores, en el modelo base se encontró que el 75.2% de los errores eran de frutas equivocadas, sin embargo en el modelo final solo 143 de los 342 errores, o sea el 41.8% corresponden a la fruta equivocada, mientras que solo 199 (58.2%) de los errores son por parte de la calidad, identificando de manera correcta a la fruta. Separando la categorización de frutas de la calidad podemos observar que el modelo identifica a la fruta correcta el 93.0% de las imágenes de prueba, por el otro lado identifica la calidad de manera correcta un 89.6%, es decir que el modelo final aprendio bien a categorizar las frutas y su problema mayor es la calidad de ellas.

Para finalizar, la confianza del promedio del modelo es de 0.958 en sus aciertos y de 0.803 en sus errores, aunque es menos confiado cuando se equivoca es una diferencia pequeña, ya que muchos de sus errores se hacen con una confianza alta.

6.3 Sesgo de captura

En la sección 3.6 se identificó que las fotografías tomadas en el 2020 fueron todas con el mismo teléfono, por lo que permite evaluar si el modelo depende de las condiciones de captura, el la tabla 6.3.1, se puede mostrar el conjunto de prueba:

Periodo	Imágenes	Exactitud
2019	1,601	88.6%
2020	417	61.6%
N/A	35	100.0%

Tabla 6.3.1 - Desempeño por período

Como se puede observar en la tabla 6.3.1 el modelo acierta el 88.6% de las imágenes en el 2019, sin embargo en el 2020 solo acierta el 61.6%, la cual es una diferencia de 27 puntos. El problema de las imágenes del 2020 son las frutas de mala calidad, con un 48.9% de exactitud, que las de buena calidad, con 71.9%. Gracias a esto confirma lo observado en validación, lo cual es que el desempeño del modelo depende de las condiciones con las que se tomaron las imágenes (telefono, fondo e iluminación) y no solo de la fruta, es por eso que cuando las condiciones cambian como en las de las imágenes del conjunto del 2020 el desempeño baja de forma considerable, es por eso que el 83.3% debe interpretarse como el desempeño esperado en las condiciones similares al dataset, y no necesariamente en cualquier entorno.

6.4 Interpretación con Grad-Cam

Dentro del análisis de Grad-Cam no formaba parte del alcance original del proyecto, sin embargo gracias a una herramienta de inteligencia artificial apoyo a la escritura del código como un paso adicional para poder interpretar las predicciones del modelo final. Sin embargo, cabe mencionar que esta herramienta no interviene en el entrenamiento ni en el funcionamiento del modelo, sino que se aplica después únicamente para visualizar qué regiones de la imagen influyen en la predicción.

Para la implementación de Grad-Cam se separó el modelo final en tres partes, la capa de normalización, la base MobileNetV2, y la cabeza de clasificación. Para cada imagen

primero se normalizan sus píxeles y después dentro de un bloque tf.Gradient.Tape registra todas las operaciones para poder calcular derivadas más adelante, la imagen pasa por MobileNetV2, para obtener sus último mapa de características y posteriormente este mapa divide la imagen en una cuadrícula de 49 zonas e indica para cada zona la intensidad de 1,280 patrones visuales aprendidos por la red. Posteriormente con tape.watch(mapas) se indica que se quiere calcular los gradientes con respecto a ese mapa y después se pasa el mapa por cada capa de la cabeza una a la vez con training=False para que el modelo se comporte igual que al predecir hasta obtener las probabilidades de las 12 clases. De ahí se toma la clase con mayor probabilidad y su valor. Fuera del bloque, tape.gradiente(puntaje, mapas) calcula cuánto cambiaría esa probabilidad si aumentara cada valor del mapa de características. Después los gradientes se promedian sobre las 49 zonas para obtener un peso de importancia para cada uno de los 1,280 patrones y después cada patrón se multiplica por su peso y se suman todos dando así un mapa de 7 x 7. Finalmente, a este mapa se le aplica la función de ReLU para conservar sólo las zonas que aumentan la probabilidad predichas y se normalizan entre 0 y 1. Para finalizar, el mapa se amplía a 224 x 224 px y se superpone sobre la imagen original con una escala de colores en donde el rojo indica que es la zona más influyente y la azul siendo la de menos.

Para entender en qué regiones de la imagen se basa el modelo se utilizó Grad-Cam o también conocido como Gradient Weighted Class Activation Mapping, esta técnica calcula el gradiente de la clase predicha respecto al ultimo mapa de características de MobileNetV2 de 7 x 7 posiciones y lo usa para generar un mapa de calor indicando cuales son las zonas de la imagen que influyen al modelo a hacer la predicción. Esto se puede notar en la figura 6.4.1



Figura 6.4.1 - Utilización de Grad-Cam en las imágenes

Como se puede observar en la figura 6.4.1, la mayoría de las imágenes muestra el mapa de calor concentrándose en la fruta y de hecho para las de mala calidad se puede observar que se concentran en las zonas con defectos visibles, como son las manchas oscuras del plátano. Gracias a esto podemos concluir que el modelo de hecho si basa sus decisiones en la fruta. En este caso se calcularon 400 imágenes de prueba para calcular la proporción del mapa de calor cae fuera del centro de la imagen. De hecho se notó que en las imágenes clasificadas correctamente la proporción estaba entre 0.53 y 0.57, lo que confirma que la atención del modelo estaba en el centro donde normalmente está la fruta, por otra parte en los casos equivocados se nota que la distribución estaba entre 0.60 y 0.62 indicando que el modelo se equivoca cuando le presta más atención al fondo.

Sin embargo los errores con mayor confianza ilustran ambos comportamientos, como se puede observar en la figura 6.4.2, se puede notar que en las naranjas de mala calidad clasificadas como buenas, el mapa de calor se centra en la fruta, lo cual sugiere que los defectos son difíciles de detectar. Por otro lado, una manzana de mala calidad fotografiada sobre un fondo azul, se puede observar como la tensión se reparte entre el fondo y las dos manzanas.Por otro lado, en los dos errores de confianza en donde clasificó mal la calidad se puede notar que el modelo se enfocó en parte al fondo para predecir, algo mencionado en la sección 3.6, y las clasificó mal debido a que las frutas de buena calidad normalmente están tomadas en condiciones similares.



Figura 6.4.2 - Errores con mayor confianza

6.5 Imágenes con calidad mixta

Como última prueba se utilizó la carpeta de calidades mixtas, la cual contiene 1,074 fotografías de frutas de buena y mala calidad, las cuales nunca fueron vistas por el modelo. Debido a que estas imágenes no tienen una sola etiqueta de calidad, solo se evaluó si el modelo identifica de manera correcta a la fruta, de hecho el modelo acertó 64.8% de los casos, lo cual es muy por debajo del 93.0% obtenido en el conjunto de prueba y su confianza promedio bajo de 0.932 a 0.790. Además clasificó como de buena calidad el 71.8% de las imágenes, lo cual era un resultado esperable debido a que el modelo fue entrenado para asignar una sola clase a cada imagen.

7 Predicciones

En esta última sección menciona que el modelo final puede generar predicciones de dos formas a partir del archivo .keras guardado, como la normalización está incluida dentro del modelo las dos reciben directamente la imagen sin procesar.

7.1 Predicciones por consola

La función predecir recibe la ruta de la imagen, la redimensiona por 224 x 224 px y muestra las tres clases más probables. En la figura 7.1.1, se puede visualizar un ejemplo:

```
predecir("/content/download.jpg")

...
download.jpg
 Apple        buena   68.6%
 Orange       mala    21.5%
 Apple        mala     8.8%
('Apple_Good', 0.6862700581550598)
```

Figura 7.1.1 - Predicción por terminal

7.2 Interfaz Gráfica

Por otro lado, se desarrolló una interfaz gráfica utilizando Gradio, la cual el usuario sube una fotografía o la puede tomar desde su celular y el modelo le asigna las tres clases más probables con sus probabilidades. En la figura 7.2.1 se puede observar el ejemplo de este método:

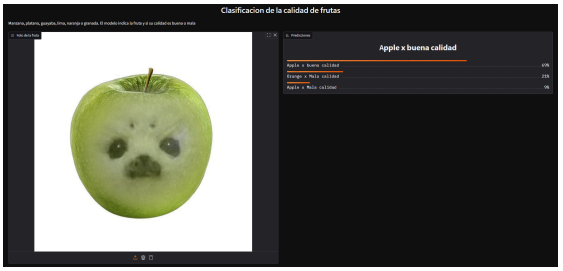


Figura 7.2.1 - Predicción por interfaz

8 Conclusión

Dentro de este trabajo se desarrolló un modelo de aprendizaje profundo el cual identificó el tipo de fruta y su calidad a partir de una fotografía, tras limpiar el dataset FruitNet y dividirlo por sesiones de captura para evitar la fuga de información, se entrenó una CNN base que mostró un sobreajuste severo (60.5%). Posteriormente, se realizaron experimentos para erradicar dicho sobreajuste, sustituyendo flatten poor Global Average Pooling 2D y aplicar transfer learning y fine-tuning con MobileNetV2 las cuales elevaron el desempeño a 90.5% enb validación y 83.3 en pruebas. El modelo final reconoce la fruta correcta en el 93.0% de los casos aunque todavía tiene dificultades con ciertas frutas. Cabe mencionar que también tiene un desempeño bajo (61.6%) dependiendo de las condiciones de captura.

9 Referencias

[1]FAO. (2019). El estado mundial de la agricultura y la alimentación 2019. Roma: FAO.

[2] Mecalux. (2023). El papel de los centros de distribución en la optimización de la cadena de suministro. Mecalux Logística.

[3] https://www.sciencedirect.com/science/article/pii/S2352340921009616

[4] https://archive.org/details/datawarehouseetl0000kimb

[5] https://archive.org/details/exploratorydataa0000tuke_7616